CS M152A Capstone Project Report
Flappy Bird FPGA Game

Nasir Hussain & Jeffrey Joseph

June 3, 2026

1 Design Specification

The project implements a hardware-based version of the Flappy Bird game on the Basys 3 FPGA board using Verilog HDL. The design integrates VGA video output, real-time game logic, user input processing, and seven-segment score display.

1.1 Input and Output Format

Inputs

- clk: 100 MHz system clock
- btn_reset: asynchronous game reset
- btn_jump: player jump control

Outputs

- vga_hsync: VGA horizontal synchronization
- vga_vsync: VGA vertical synchronization
- vga_r[3:0], vga_g[3:0], vga_b[3:0]: VGA color outputs
- seg[6:0]: seven-segment display segments
- an[3:0]: seven-segment display anodes
- dp: decimal point control
- led[2:0]: game state indicators

1.2 Functional Behavior

- The player controls a bird that moves vertically through a scrolling obstacle field.
- Pressing the jump button causes the bird to gain upward velocity.
- Gravity continuously accelerates the bird downward.
- Pipes move horizontally across the screen and are recycled once they leave the display area.

- The score increments whenever the bird successfully passes a pipe.
- Collision with pipes, the ground, or the top boundary results in a game-over condition.
- The current score is displayed on the Basys 3 seven-segment display.

2 Block-Level Architecture

The system is composed of five primary Verilog modules.

2.1 flappy_top

This module serves as the top-level integration block.

- Generates pixel timing from the system clock.
- Synchronizes button inputs.
- Produces single-cycle jump pulses.
- Generates a slower game update clock using frame division.
- Instantiates all game subsystems.

2.2 vga_timing

This module generates standard VGA timing signals.

- Produces horizontal and vertical synchronization pulses.
- Tracks current pixel coordinates.
- Generates frame timing events.
- Supports 640×480 VGA output.

2.3 flappy_game_core

This module implements the primary game mechanics.

- Bird position and velocity updates.
- Gravity and jump physics.
- Pipe movement and recycling.
- Collision detection.
- Score tracking.
- Game state transitions:
  - Start State
  - Playing State
  - Game Over State

  Pseudo-random pipe positions are generated using an LFSR-based random number generator.

2.4 flappy_renderer

This module converts game-state information into pixel colors.

- Draws the background.
- Draws the bird sprite.
- Draws obstacle pipes.
- Draws the ground.
- Produces RGB values for VGA output.

2.5 score_display

This module controls the seven-segment display.

- Displays the current score.
- Uses multiplexed digit scanning.
- Interfaces with the segment decoder.

3 Verilog Implementation Files

- Design: flappy_top.v
- Game Logic: flappy_game_core.v
- VGA Timing: vga_timing.v
- Graphics Renderer: flappy_renderer.v
- Score Display: score_display.v
- Seven-Segment Decoder: seg7_decoder.v
- Testbench: flappy_game_core_tb.v
- Constraints: Basys3.xdc

4 Verification Methodology

The design was verified through both simulation and FPGA hardware testing.

- VGA synchronization timing was verified using simulation waveforms.
- Bird movement was validated by observing gravity and jump responses.
- Pipe scrolling behavior was checked for continuous obstacle generation.
- Collision detection was tested against pipe boundaries, ceiling, and ground conditions.
- Score increments were verified when the bird successfully passed obstacles.
- Seven-segment score display was verified using multiple score values.

5 Representative Test Cases

Test Condition	**Input Action**	**Expected Output**	**Result**
Reset	btn_reset pressed	Game returns to initial state	Pass
Game Start	btn_jump pressed	Transition to Playing State	Pass
Jump Action	btn_jump during gameplay	Bird moves upward	Pass
Gravity Update	No button input	Bird falls downward	Pass
Pipe Motion	Frame updates active	Pipes move left across screen	Pass
Score Increment	Bird passes pipe	Score increases by one	Pass
Ground Collision	Bird reaches ground	Game Over asserted	Pass
Pipe Collision	Bird intersects pipe	Game Over asserted	Pass
Display Update	Score changes	Seven-segment display updates	Pass

Table 1: Flappy Bird functional test cases

6 Conclusion

A fully functional FPGA implementation of the Flappy Bird game was developed using Verilog HDL on the Basys 3 platform. The design integrates VGA graphics, real-time game physics, obstacle generation, collision detection, score tracking, and seven-segment display output. Modular design practices were used to separate timing generation, game logic, rendering, and display control. Simulation and hardware testing confirmed correct operation of all major subsystems. The project demonstrates practical application of synchronous digital design, finite state machines, VGA video generation, and FPGA-based interactive game development.